

Keeping Meaning Aligned on a Budget: Drift Detection and Realignment for AI-Native Communication

Abstract—This document is a complete definition and execution roadmap for a Research Project aligned with the companion paper on temporal knowledge drift in semantic communication. The roadmap specifies the scientific gap, a simplified system model, a methodology, an eight-experiment plan, baselines, metrics, deliverables, and an explicit mapping from each student result to a figure or theorem of the companion paper.

Index Terms—Semantic communication, representation drift, concept drift, representation alignment, continual learning, drift detection, resource-aware communication, AI-native 6G.

I. THESIS ABSTRACT

Semantic communication (SemCom) lets two endpoints exchange task-relevant meaning through a shared learned representation, a codebook, a set of prototypes, or a jointly trained encoder-decoder pair, instead of exchanging raw bits. Almost all current systems assume this shared representation is fixed and aligned after deployment. In practice, the transmitter and the receiver are separate AI models that keep learning, each from its own local data stream. Because those streams differ and change over time, the two models update independently and asymmetrically, and the representation that was aligned at deployment slowly diverges: the same code or latent direction no longer means the same thing at both ends. We call this temporal representation drift, and its hallmark is that task accuracy degrades even when no information is lost in transmission—the failure is one of shared meaning, not of delivery.

The straightforward fix—periodically resending the full model or knowledge base—restores alignment but is expensive in exactly the resource SemCom is meant to save (bandwidth, energy, latency). This work investigates a lightweight alternative. We build an AI experimental framework in which a transmitter encoder and a receiver decoder, initially aligned, drift apart under independent local updates; we compare a family of drift metrics for how well each predicts the resulting task degradation; we implement a receiver-side drift detector that fires from observable fidelity signals without seeing the transmitter’s state; and we implement low-overhead realignment operators (prototype exchange, anchor-based relative-representation alignment / Procrustes maps, and small adapters) that re-synchronize meaning using only a tiny payload. The channel is modeled abstractly—as a message-size budget, embedding noise, feature dropout/erasure, and a separate alignment budget—so the work stays focused on the AI questions while remaining faithful to wireless constraints.

II. INTRODUCTION

A. Semantic communication, in machine-learning terms

If you have built a learned image or text compressor, you already understand most of semantic communication. An ordinary compressor learns an encoder that maps an input to a short code and a decoder that reconstructs the input. Semantic communication keeps the encoder-decoder idea but changes the goal: the decoder does not have to reconstruct the input pixel-for-pixel; it only has to recover whatever is needed for a *downstream task*—a class label, an answer to a question, a control decision. The encoder learns to keep task-relevant information and discard the rest, and the “code” it sends is a compact embedding or a sequence of codebook indices.

There is one extra requirement that does not appear in ordinary compression: *the code has to mean the same thing to both sides*. In a standard autoencoder the encoder and decoder are trained together and shipped together, so this is automatic. In a communication system the encoder lives on one device (the *transmitter*) and the decoder on another (the *receiver*), and they must share a common understanding of what each symbol denotes. That shared understanding is the system’s *knowledge base* or *semantic codebook*.

B. Why future networks might send meaning instead of bits

Future (6G) networks are expected to connect enormous numbers of bandwidth-, power-, and latency-constrained devices that cooperate on inference rather than on raw data transport—cameras feeding a recognizer, sensors feeding a controller, agents coordinating a task. Sending a 200-byte task-sufficient embedding instead of a 2-MB image is an obvious win when the receiver only needs the label. This is why SemCom is studied as a building block of “AI-native” 6G: the air interface starts to carry *learned representations* rather than generic bytes.

C. The problem this thesis attacks

Here is the catch that motivates the whole thesis: *deployed models are not frozen*. The transmitter’s encoder keeps adapting to the data it sees; the receiver’s decoder keeps adapting to the data it sees; and those data are not the same. After enough independent updates, the two sides no longer agree on meaning. Concretely: the transmitter encodes a “dog” image into a region of latent space; the receiver, having fine-tuned on a different stream, now interprets that region as something slightly different; nothing went wrong on the

wire—the embedding arrived perfectly—but the prediction is wrong anyway. This is temporal representation drift, the central object of the work.

A naive remedy is to periodically retransmit the whole model so both sides reset to a common point. But modern encoders and codebooks are large, and resending them repeatedly burns the very bandwidth, energy, and latency that SemCom was supposed to save; it may also be impossible under intermittent connectivity or privacy constraints. So the real question is sharper: *can the receiver notice that meaning has drifted, and can the two endpoints re-agree, using only a tiny amount of communication?*

D. Where AI comes in

This problem sits at the intersection of four topics the student already knows or can pick up quickly:

- **Representation learning**—drift is a phenomenon in *embedding space*, so we measure it with representation-similarity tools (cosine/prototype distance, CKA, Procrustes).
- **Continual learning**—the independent local updates are exactly continual-learning processes, with the familiar tension between adapting and forgetting (e.g., EWC-style regularization).
- **Distribution shift / concept drift**—the receiver-side detection problem is a streaming change-detection problem (Page–Hinkley, CUSUM, ADWIN).
- **Representation alignment / model stitching**—the realignment primitives (relative representations against shared anchors, linear/Procrustes maps, small adapters) are the AI-native way to make two independently trained latent spaces talk to each other.

The thesis ties these together inside a communication setting, where the alignment must happen *on a budget*. That budget is the only place the wireless world enters, and we keep it simple.

Key framing. *This is not a single model adapting to a moving world; it is two models drifting apart from each other, and the question is when and how cheaply to bring them back together.*

III. PROBLEM STATEMENT

A. Central research question

*In a transmitter–receiver semantic communication system whose two endpoints update their representations independently over time, how can the resulting representation drift be **detected** from receiver-observable signals and **corrected** using only a small, budget-limited amount of alignment information—rather than full model or knowledge-base synchronization?*

B. Sub-questions

- 1) **Measurement.** How do we quantify drift between the transmitter’s and receiver’s representations, given that neither side can see the other’s parameters directly?
- 2) **Cause.** How does independent local continual learning at each endpoint create semantic mismatch over time, and how do drift type and speed shape it?
- 3) **Consequence.** How much, and how quickly, does drift degrade downstream task accuracy on an otherwise error-free link?
- 4) **Early warning.** Can drift be detected from a receiver-side proxy signal (confidence, consistency, windowed loss) *before* task accuracy collapses, and with what detection delay and false-alarm rate?
- 5) **Correction.** Can exchanging only anchors, prototypes, a low-rank map, or a small adapter restore alignment without sending the full model?
- 6) **Cost.** How much alignment overhead (bits / number of exchanged items) is actually needed to recover most of the lost accuracy?
- 7) **Metric selection.** Which drift metric best predicts semantic misunderstanding, i.e., correlates most strongly with task degradation and detects it earliest?

IV. SCIENTIFIC CONTRIBUTION

- 1) An AI experimental framework for bilateral representation drift in a transmitter–receiver SemCom setting: an initially aligned encoder/decoder pair whose two sides update independently, with a clean abstract communication layer.
- 2) A controlled drift benchmark in which transmitter and receiver representations evolve independently under reproducible drift scenarios (corruption/domain drift, class-incremental drift, label/feature shift), with tunable drift type and speed.
- 3) An empirical comparison of drift metrics (prototype distance, Procrustes error, CKA, codeword-usage divergence, task-disagreement rate) ranked by how well each predicts and how early each detects task degradation.
- 4) A lightweight, receiver-side drift detector built on observable fidelity/task signals (Page–Hinkley / CUSUM / ADWIN on a confidence, consistency, or windowed-loss signal), characterized by detection delay, false-alarm rate, and missed-detection rate.
- 5) Lightweight realignment operators—prototype/partial-codebook exchange, anchor-based relative-representation alignment and Procrustes maps, and small adapters—that modify only the receiver side and use a payload far smaller than the full model.
- 6) An empirical overhead-vs-recovery characterization: Pareto curves of task accuracy against alignment bits, and event-triggered versus periodic realignment at matched overhead.
- 7) A reusable, reproducible pipeline (code, configs, logged runs, plotting scripts) and a results summary formatted for direct inclusion in the IEEE paper.

V. SYSTEM MODEL

A. Transmitter and receiver

We consider a point-to-point link over discrete rounds $t = 1, 2, \dots, T$. At round t , the transmitter T observes input $X_t \in \mathcal{X}$ with task target $Y_t \in \mathcal{Y}$, extracts a semantic representation, and sends a compact message; the receiver R interprets the message and predicts $\hat{Y}_t$. The round- t task loss is $\ell(\hat{Y}_t, Y_t)$, and semantic task accuracy is the fraction of rounds with $\hat{Y}_t = Y_t$. Crucially, the two endpoints' local data follow non-identical, time-varying distributions: T updates on $p_{T,t}(x, y)$ and R on $p_{R,t}(x, y)$, with $p_{T,t} \neq p_{R,t}$ in general.

B. Semantic encoder and decoder

The transmitter encoder produces a latent

$$z_t = f_{T,t}(X_t; K_{T,t}) \in \mathbb{R}^d, \quad (1)$$

where $f_{T,t}$ is the transmitter-side encoder, parameterized by its knowledge state $K_{T,t}$. The receiver decodes the received message

$$\hat{Y}_t = g_{R,t}(\tilde{z}_t; K_{R,t}), \quad (2)$$

where $g_{R,t}$ is the receiver-side decoder/classifier and $\tilde{z}_t$ is the (possibly noisy, quantized, or partially erased) received representation.

Model choices, simplest to strongest:

- **Minimal viable (recommended start):** a frozen pretrained backbone (e.g., ResNet-18) followed by a small trainable projection head, plus a set of *class prototypes*. The prototype set is the semantic codebook: atom q is the prototype of concept q . This is the easiest case to reason about, because drift is literally “the prototypes move.”
- **Discrete-codebook variant:** a small vector-quantized (VQ) codebook $\{c^{(1)}, \dots, c^{(Q)}\}$. The encoder quantizes z_t to nearest atoms; the message is the atom indices. This is the truest “semantic codebook” and matches the paper most directly.
- **Stronger optional variants:** ViT-Tiny / ViT-Small; a CLIP-style frozen encoder with a trainable adapter; a small transformer for the text task. Defer these until the prototype/VQ pipeline works end to end.

C. Local knowledge states

Each endpoint maintains a knowledge state $K_{T,t}, K_{R,t}$, abstracting encoder/decoder parameters, the semantic codebook $\{c^{(q)}\}$, class prototypes, adapter weights, or an embedding space. *Alignment at deployment* means $K_{T,0} \approx K_{R,0}$ (atom q denotes the same concept at both ends); *drift* is the divergence of these states over t .

D. Independent local updates

Between rounds, each side updates from its own freshly observed local data:

$$K_{T,t+1} = \mathcal{U}_T(K_{T,t}, \mathcal{D}_{T,t}), \quad (3)$$

$$K_{R,t+1} = \mathcal{U}_R(K_{R,t}, \mathcal{D}_{R,t}), \quad (4)$$

where $\mathcal{D}_{T,t} \sim p_{T,t}$, $\mathcal{D}_{R,t} \sim p_{R,t}$, and $\mathcal{U}_T, \mathcal{U}_R$ are continual-learning updates (fine-tuning, rehearsal, or regularized updates such as EWC). These updates are applied *independently*—neither side sees the other's data—and the two local datasets may be non-IID and time-varying. This independence produces drift, and it distinguishes the setting from federated learning (no coordinator, no averaging toward a common point).

E. Semantic drift metric

Drift at round t is the discrepancy between the two states, or between the representations they induce on a probe input X :

$$\Delta_t = d(K_{T,t}, K_{R,t}) \quad \text{or} \quad \Delta_t = d_{\text{rep}}(f_{T,t}(X), f_{R,t}^{\text{rep}}(X)), \quad (5)$$

where $f_{R,t}^{\text{rep}}$ is the receiver's *reference* representation. The framework is metric-agnostic; the student tests several:

- 1) Cosine distance between matched class prototypes.
- 2) Euclidean (prototype) distance, $\frac{1}{Q} \sum_q \|c_{T,t}^{(q)} - c_{R,t}^{(q)}\|$.
- 3) Procrustes alignment error, $\min_{\Phi \in O(d)} \|Z_T \Phi - Z_R\|_F$ over stacked probe representations.
- 4) CKA distance, $1 - \text{CKA}(Z_T, Z_R)$.
- 5) KL divergence between predicted class distributions, or between codeword-usage distributions $\text{KL}(P_{T,t}(q) \| P_{R,t}(q))$.
- 6) Task-disagreement rate (fraction of probes where the two pipelines predict differently).
- 7) Semantic-fidelity degradation (drop in a reconstruction/consistency score).
- 8) Topographic-similarity degradation (optional; useful if a codebook is used).

A note worth recording: prototype distance and task-disagreement rate are the *most communication-relevant*—the former is tied to the realignment payload, the latter to the quantity being optimized.

F. Abstract communication and alignment budget

This is the only part of the model that stands in for wireless, and it is intentionally small. There are *two channels*: one carries the task message, the other carries alignment information.

Task message (per round):

$$m_t = q_B(z_t), \quad \tilde{z}_t = q_B(z_t), \quad (6)$$

where $q_B(\cdot)$ degrades the representation according to a message budget B . The student can realize q_B as any combination of bit-rate quantization (fewer bits per coordinate), top- k feature retention with the rest dropped, additive Gaussian noise of variance set by a notional SNR, or random feature erasure.

Alignment message (only when realignment is triggered):

$$A_t, \quad |A_t| \leq B_{\text{align}}, \quad (7)$$

where A_t may contain a few anchor encodings, prototypes, calibration embeddings, low-rank map coefficients, or a small adapter. B_{align} is a limited synchronization budget.

Table I maps every abstract knob back to its wireless meaning. It is the single most important thing keeping the work communication-meaningful while the student never touches modulation, coding, OFDM, MIMO, or fading theory.

G. Realignment function

When triggered, realignment modifies the *receiver side only*, so that R decodes in T 's current frame:

$$\tilde{K}_{R,t} = \mathcal{R}(K_{R,t}, A_t) \quad \text{or} \quad \tilde{g}_{R,t} = g_{R,t} \circ \mathcal{T}_t, \quad (8)$$

where $\mathcal{T}_t$ is a learned or estimated alignment map (e.g., a Procrustes/linear map fit from anchor pairs, or a small adapter). The cost of realignment is $B_{\text{align},t} = |A_t|$.

H. End-to-end objective

The per-round empirical loss is

$$\mathcal{L}_t = \mathbb{E}_{(X_t, Y_t)} [\ell(g_{R,t}(q_B(f_{T,t}(X_t))), Y_t)]. \quad (9)$$

The realignment-aware objective the thesis studies (with $\delta_t \in \{0, 1\}$ a realignment trigger and $D_{\text{detect},t}$ the detection delay charged to round t) is

$$\min \sum_{t=1}^T \mathcal{L}_t + \lambda \sum_{t=1}^T \delta_t B_{\text{align},t} + \mu \sum_{t=1}^T D_{\text{detect},t}. \quad (10)$$

In one line: keep $\sum_t \mathcal{L}_t$ low while spending as few alignment bits as possible—and decide *when* spending them is worth it. (In the paper this objective also carries energy and channel-use terms; the student's λ, μ version is its simplified shadow.)

VI. DRIFT DETECTION AND REALIGNMENT FORMULATION

A. The detection problem

The receiver *cannot observe* Δ_t *directly*—it never sees $K_{T,t}$. So it monitors an observable proxy signal S_t expected to move as drift grows: a moving average of prediction confidence (max softmax probability); a consistency score (agreement under light augmentations, or prediction stability across a short window); self-supervised reconstruction fidelity on acknowledged samples; or a windowed task loss when sparse labels are occasionally available. A first, necessary experiment is simply whether S_t actually tracks Δ_t and the task loss; if no observable proxy moves with drift, online detection is impossible and the signal choice must be revisited.

B. A simple drift detector

Run a sequential change detector on S_t . A Page–Hinkley / CUSUM test accumulates deviations from the aligned baseline S_0 :

$$g_t = \max\{0, g_{t-1} + (S_t - S_0 - \kappa)\}, \quad \delta_t = \mathbf{1}\{g_t > h\}, \quad (11)$$

with slack κ and threshold h ; ADWIN (available in the `river` library) is a drop-in alternative. In the paper's threshold form,

$$\delta_t = \begin{cases} 1, & \hat{\Delta}_t > \tau, \\ 0, & \text{otherwise,} \end{cases} \quad (12)$$

where $\hat{\Delta}_t$ is the estimated drift and τ a threshold. The detector is characterized by four quantities to log and report: false-alarm rate (P_{FA}), missed-detection rate (P_{MD}), detection delay (D_{detect}), and the resulting realignment overhead (wasted alignment bits on spurious triggers). Lowering h shortens delay but raises false alarms—this monotone trade-off is itself a result.

C. Realignment methods to implement

- 1) **Prototype exchange**—send the few most-drifted class prototypes; R overwrites its own.
- 2) **Anchor / relative-representation exchange**—agree on a fixed anchor set $\Omega = \{\omega_1, \dots, \omega_M\}$; T sends its encodings of the anchors; R re-expresses its space relative to those shared anchors (relative representations enable training-free communication between independent latent spaces, the most AI-native primitive here).
- 3) **Calibration-set alignment**—send a small calibration batch and re-fit the receiver head.
- 4) **Linear / Procrustes alignment**—from anchor pairs (Z_T, Z_R), estimate Φ (orthogonal or low-rank, $O(dr)$ coefficients) that maps R 's space onto T 's.
- 5) **Adapter update**—train a small adapter (a few thousand parameters) on the alignment payload, in place of full weights.
- 6) **Partial codebook correction**—overwrite only the drifted atoms of a VQ codebook.

In all cases the correction touches the receiver only, and the payload $|A_t| \ll |K|$.

D. Cost-aware triggering (the part that makes it communication)

Detection answers *whether* drift is present. The trigger must also decide *whether realignment is worth its communication cost*. The companion paper proves a threshold of the form

$$\tau_t \propto \frac{C_{\text{align}}}{L_z(1 - \rho)}, \quad (13)$$

i.e., realign only when the estimated drift-induced loss exceeds the realignment cost (C_{align} the alignment cost, L_z the task sensitivity, ρ the residual-drift fraction after realignment). The student does not derive this; the student *demonstrates it empirically*: a noisier or more expensive channel should make the best-performing trigger fire later or demand more budget. That demonstration is one of the most valuable plots for the paper.

VII. METHODOLOGY

A. Dataset options

Table II lists candidate datasets. **Recommended first option:** CIFAR-10 with CIFAR-10-C corruptions for the easiest, most controllable drift, and CIFAR-100 for class-incremental drift, which gives the most convincing semantic story. Start on CIFAR-10/10-C; add CIFAR-100 once the pipeline is stable.

TABLE I
MAPPING OF ABSTRACT KNOBS TO THE WIRELESS QUANTITIES THEY REPRESENT.

Abstract knob in the thesis	Wireless quantity it represents
Message budget B (bits / retained features)	Bandwidth; number of channel uses
Embedding noise variance / notional SNR	Channel noise; transmit power; SNR
Feature dropout / message erasure	Fading; packet loss; deep-fade erasure
Quantization level	Digital transmission with a finite codebook
Alignment budget B_{align}	Synchronization overhead the link can afford
Realignment frequency / trigger delay	Latency and energy constraints
Small payload, never the full model	Edge-device resource and connectivity limits

TABLE II
DATASET OPTIONS FOR THE DRIFT BENCHMARK.

Dataset	Use	Difficulty	Value for the paper
CIFAR-10 + CIFAR-10-C CIFAR-100	corruption / domain drift of rising severity	low	high—clean, standard, controllable
TinyImageNet DomainNet subset MNIST / Fashion-MNIST	class-incremental drift (asymmetric class introduction) larger-scale domain shift strong real domain shift debugging only	low—medium medium medium—high trivial	high—most “semantic” drift medium medium low (do not report as a main result)
Evolving-domain text classification	concept drift with a language encoder	medium	medium (optional second modality)

B. Drift generation

Generate drift by *asymmetric fine-tuning*: start from an aligned pair, hold one endpoint fixed while the other fine-tunes on a shifting local stream (then symmetrically), so Δ_t grows monotonically between realignments. Drift *speed* is controlled by the per-round update magnitude and the rate of distribution change. Scenarios to support: corruption/domain drift (CIFAR-10-C, rising severity)—*easiest to debug*; class-incremental drift (new classes appear asymmetrically)—*most useful for the paper’s semantic narrative*; label-distribution shift; feature-distribution shift; the underlying asymmetric T/R fine-tuning mechanism; non-IID local updates; and gradual vs sudden drift (gradual stresses the detector’s delay, sudden stresses its sensitivity). **Recommendation**: establish the pipeline on gradual corruption drift; report the headline results on class-incremental drift.

C. Model options

Minimal viable setup: frozen ResNet-18 backbone + small projection head + class prototypes (the prototypes are the codebook). Cheap, fast, and drift is directly visible as prototype movement. **Stronger setup**: add a VQ codebook and/or swap the backbone for ViT-Tiny or a CLIP-style frozen encoder with a trainable adapter. Use these only after the minimal setup produces clean curves.

D. Alignment methods to implement

No alignment (lower-bound reference); full synchronization (upper-bound reference); periodic prototype exchange; event-triggered prototype exchange; Procrustes / relative-representation alignment from anchors; adapter-based realign-

ment; and oracle alignment (upper bound assuming a perfect detector / perfect map).

E. Training protocol

- **Splits**: standard train/validation/test, plus a held-out *probe set* (drift measurement only) and a small *anchor set* Ω shared by both endpoints.
- **Shared initialization**: train one model jointly (or from a common checkpoint) so $K_{T,0} \approx K_{R,0}$; record the aligned baseline accuracy L_0 .
- **Local updates**: alternate (or interleave) transmitter-side and receiver-side fine-tuning on their respective streams.
- **Drift schedule**: fix a reproducible per-round update magnitude and distribution-shift rate.
- **Threshold selection**: choose detector thresholds on validation drift, then freeze before testing (no peeking at test-set drift).
- **Budget sweep**: vary B_{align} (number of anchors/prototypes/adapters size) over a documented grid.
- **Seeds**: at least 3–5 random seeds per configuration; report mean and standard deviation. This is non-negotiable for a credible result.
- **Logging and hyperparameters**: log every metric per round to disk; fix and record all hyperparameters in config files.

F. Drift-metric evaluation

Compare drift metrics on: correlation with task-accuracy degradation (the primary criterion); detection delay when each metric is fed to the detector; false-alarm and missed-detection rates; robustness to message noise; and the com-

pute/communication cost to evaluate the metric (a metric that needs many probe exchanges is itself expensive).

VIII. EXPERIMENTAL PLAN

Eight experiments match the paper, grouped into priority tiers.

A. Tier 1—core (must complete)

Experiment 1—Drift without realignment. *Objective:* establish the premise—independent updates cause drift that degrades task accuracy on an error-free link. *Variables:* round t ; drift type; channel noise off. *Plots:* accuracy vs t ; Δ_t vs t (each metric); representation similarity vs t . *Supports if:* accuracy falls and Δ_t rises together while noise is off (drift, not channel, is the culprit). *Weakens if:* Δ_t stays flat or accuracy is unaffected—drift too weak; increase update magnitude / shift rate.

Experiment 2—Drift-metric comparison. *Objective:* find which metric best predicts and earliest detects degradation. *Variables:* drift metric; detector threshold. *Outputs:* metric-vs-task-loss correlation; detection ROC (P_{FA} vs P_{MD}); false-alarm/missed-detection table; delay vs threshold. *Supports if:* at least one observable proxy correlates strongly with task loss and yields a usable ROC. *Weakens if:* no proxy tracks the loss—revisit S_t .

Experiment 3—Lightweight realignment vs baselines. *Objective:* show that a small payload recovers most of the lost accuracy. *Variables:* alignment method (no-align, full-sync, prototype, anchor/Procrustes, adapter). *Plots:* accuracy recovery vs t ; alignment overhead (bits); performance—overhead trade-off. *Supports if:* lightweight methods track the full-sync upper bound while sending far fewer bits. *Weakens if:* only near-full-sync overhead approaches the upper bound.

B. Tier 2—important (complete if on schedule)

Experiment 4—Event-triggered vs periodic realignment. *Objective:* show triggering beats fixed-period sync at equal overhead. *Plots:* accuracy vs t ; number of realignment events; total alignment cost. *Supports if:* at matched bits, the trigger holds higher accuracy. *Weakens if:* periodic matches the trigger—drift too regular for triggering to help (note this honestly; it bounds the contribution).

Experiment 5—Alignment-budget sweep. *Objective:* find how recovery scales with payload size. *Variables:* number of anchors/prototypes/adapter size. *Plots:* accuracy recovery vs B_{align} ; drift reduction vs B_{align} (a Pareto front). *Supports if:* accuracy saturates well below full-sync size (a “knee”). *Weakens if:* recovery keeps needing more bits up to full-sync.

C. Tier 3—stretch (time-permitting; high value for the paper)

Experiment 6—Drift-severity sweep. Vary drift speed and type; plot robustness under mild/moderate/severe drift and detection delay vs drift speed.

Experiment 7—Abstract channel-noise sensitivity. Turn on embedding noise / dropout / erasure on the task and alignment messages; plot realignment success and detection

reliability vs noise level. *This experiment is where the channel abstraction earns its keep—the empirical stand-in for the paper’s SNR-sensitivity figure and the channel-aware trigger.*

Experiment 8—Generalization to unseen drift. Fit thresholds / alignment rules on one drift type, test on another; report cross-drift transfer.

IX. BASELINES

Table III lists baselines and whether each is mandatory. Baseline 10 deserves emphasis: comparing channel-aware vs channel-unaware triggering is precisely what shows the cost-aware decision matters, so it is treated as mandatory even though a naive reading might call it optional.

X. METRICS

Log all of these per round, per seed: task accuracy over time; cross-entropy loss over time; semantic drift magnitude Δ_t (each metric); representation similarity; prototype distance; CKA; task-disagreement rate; detection delay; false-alarm rate; missed-detection rate; alignment overhead (bits, and number of items exchanged); number of realignment events; recovery time after a realignment; *area under the degradation curve* (a single scalar summarizing how much accuracy was lost over the run—very useful for ranking methods); compute cost; and robustness to noise/dropout.

XI. RESULTS SECTION

- 1) **Temporal drift without realignment.** *Insert:* accuracy-vs-time and Δ_t -vs-time figures (Exp 1). *Interpret:* whether accuracy degrades while the channel is error-free, and whether Δ_t rises in step—this establishes (or refutes) the premise.
- 2) **Drift-metric validity.** *Insert:* metric-vs-loss correlation table and detection ROC (Exp 2). *Interpret:* name the best-predicting and earliest-detecting metric.
- 3) **Realignment performance.** *Insert:* recovery curves for all methods (Exp 3). *Interpret:* how close each lightweight method gets to the full-sync upper bound, and at what payload.
- 4) **Event-triggered vs periodic.** *Insert:* accuracy-vs-time and event-count comparison at matched overhead (Exp 4). *Interpret:* whether triggering wins, and by how much.
- 5) **Alignment-overhead analysis.** *Insert:* accuracy-vs-bits Pareto curve (Exp 5). *Interpret:* locate the knee—the cheapest payload that recovers most accuracy.
- 6) **Drift-severity analysis.** *Insert:* robustness across drift speeds/types (Exp 6). *Interpret:* the regime where the method holds and where it breaks.
- 7) **Robustness to noisy alignment messages.** *Insert:* success-vs-noise curves (Exp 7). *Interpret:* how channel noise shifts the cost-aware operating point.
- 8) **Discussion.** *Insert:* synthesis. *Interpret:* which sub-questions were answered, with what confidence, and what the limitations are.

TABLE III
BASELINES AND THEIR ROLE. “MANDATORY” MARKS THE COMPARISONS NEEDED FOR THE PAPER’S HEADLINE CLAIMS.

#	Baseline	Role	Mandatory?
1	No realignment	lower bound	Mandatory
2	Full synchronization	upper bound	Mandatory
3	Periodic full synchronization	overhead reference	Mandatory
4	Periodic prototype exchange	lightweight periodic reference	Mandatory
5	Random-anchor exchange	sanity / anchor-choice ablation	Optional
6	Fixed shared encoder, no local learning	“drift can’t happen” reference	Recommended
7	Local learning without drift control	cost of doing nothing while learning	Recommended
8	FedAvg-style periodic averaging	contrast with the federated paradigm	Optional
9	Oracle drift detector	upper bound on detection	Recommended
10	Channel-unaware realignment	isolates the value of cost-aware triggering	Mandatory

XII. CONCLUSION

To be filled later.

XIII. IMPLEMENTATION ROADMAP

A. Build order

- 1) Implement and train a baseline classifier on CIFAR-10; record clean accuracy.
- 2) Wrap it into a transmitter encoder + receiver decoder with a prototype (or VQ) codebook.
- 3) Create a shared aligned initialization ($K_{T,0} \approx K_{R,0}$); record the aligned baseline L_0 .
- 4) Implement the abstract communication layer (q_B : quantize/noise/dropout/erasure)—a small, self-contained module.
- 5) Inject independent local drift via asymmetric fine-tuning; verify Δ_t grows (Exp 1).
- 6) Implement the drift metrics and the receiver-side signal S_t .
- 7) Implement the no-realignment lower bound and the full-sync upper bound.
- 8) Implement lightweight realignment (prototype, then anchor/Procrustes, then adapter).
- 9) Implement the event-triggered detector (Page–Hinkley/CUSUM/ADWIN).
- 10) Add the noisy-alignment-message path for the channel-sensitivity experiment.
- 11) Run the drift and realignment experiments across seeds and budgets.
- 12) Generate plots and the area-under-degradation-curve summaries.
- 13) Write the thesis results and the paper-ready results summary.

XIV. EXPECTED DELIVERABLES

A code repository (documented, with a README and environment file); experiment configuration files (one per reported figure, so every result is reproducible from a config); trained model checkpoints for the aligned initialization and

key runs, if storage allows; raw logs (per-round metrics, all seeds); plotted results as both figures and the underlying data files; drift-metric scripts and alignment-method scripts as standalone reusable modules; the final thesis report; reproducibility instructions (exact commands to regenerate each figure); and a results summary formatted for the IEEE paper, with figures whose captions match the paper’s figure list plus a short text block reporting the observed patterns.